Voice AI Agent Assignment

Models · Domain Customization · Evaluations

A 3-week hands-on assignment for building and evaluating a domain-specific AI phone agent.

Part 0 — Introduction

What you are building

You will design, configure, and evaluate an **AI voice phone agent** for one real-world domain (for example: clinic appointment booking, lead qualification, order status, or payment reminders).

An AI voice agent is **not** a chatbot in a browser and **not** a dictation app (like tools that type what you say into a document). It is a system that:

1. **Listens** to a caller on a phone line (speech → text)
2. **Reasons** about what to do next (language model + business rules)
3. **Speaks** back naturally (text → speech)
4. **Acts** through tools (look up records, book slots, escalate to a human)

You will **not** train neural networks from scratch. You will **select, configure, and compare** existing speech and language services, then prove quality with structured tests.

The three parts of this assignment

Part	Week	Weight	What you prove
1 — Models	1	30%	You can choose and compare STT, LLM, and TTS for phone calls
2 — Domain customization	2	35%	You can configure prompts, tools, and scenarios for one use case
3 — Evaluations	3	35%	You can measure agent quality with golden tests and pass thresholds

What to submit

Submit **one portfolio PDF** plus a **folder or Git repo** containing:

- All three part reports (2–3 pages each)
- Model comparison tables and audio samples (if any)
- Your agent configuration files and test scenarios
- Test run outputs (logs, tables, screenshots)

Suggested demo: 5–8 minute walkthrough of model choices → agent behavior → test results.

Part 1 — Voice Models (Week 1)

1.1 Learning goals

By the end of Part 1 you should be able to:

- Name the four model types used in voice agents and what each does
- Explain why phone audio needs different testing than clean studio recordings
- Compare two speech-to-text systems on domain-specific utterances
- Compare two text-to-speech voices for clarity and latency
- Estimate cost per call for your chosen stack

1.2 The four model types

Every voice phone agent uses some combination of these:

Type	Also called	Input → Output	Job on the call
STT	ASR, speech-to-text	Audio → text	Hear what the caller said
LLM	Large language model	Text + tools → text + tool calls	Decide what to say and which actions to take
TTS	text-to-speech	Text → audio	Speak the agent's reply
S2S	Speech-to-speech, realtime voice	Audio ↔ audio	Combined listen-and-speak in one model

Important rules:

- Only the **LLM** (or S2S brain) should decide **side effects** (booking, charging, deleting data).
- STT and TTS are **transducers** — they convert formats; they should not make business decisions.
- **Dictation apps** (speech → text into Notes or email) are **not** phone agents. They have no call loop, no tools, and no turn-taking with a remote caller.

1.2.1 Streaming vs batch speech recognition

Mode	How it works	When to use
Streaming STT	Partial transcripts while the caller is still speaking	**Live phone calls** (required for good UX)
Batch STT	Waits for full recording, then transcribes	Post-call notes, compliance archives

Do not put slow batch transcription on the live call path unless you have measured latency and proven it is acceptable.

1.3 Choose your domain use case

Pick **one** domain scenario. Examples:

- Inbound clinic appointment booking
- Outbound payment reminder
- Inbound order status lookup

- Outbound lead qualification before demo booking

Write a **half-page problem statement**: who calls, why, and what "success" means (e.g., "appointment booked," "payment promised," "escalated to human").

1.4 Model selection matrix

Research at least **two options per layer**. Fill in your choices:

Layer	Option A	Option B	**Your choice**	Why (latency, language, cost, quality)
STT				
LLM				
TTS				
Fallback (STT or TTS)				

You may use free tiers, trial APIs, or **mock/simulated** providers for early work. State clearly what you would use in production.

1.5 STT hands-on comparison

Create **5 utterances** a real caller might say in your domain. Each must include at least one **hard entity** (person name, date, phone number, order ID, or amount).

Example (clinic booking):

1. "I'd like to book with Dr Sharma next Tuesday at four pm"
2. "My patient ID is P nine four seven two one"
3. "Can you move my appointment from March fifth to March twelfth?"
4. "Call me back on nine eight seven six five four three two one zero"
5. "I'm not sure of the spelling — it's Ananya, A-N-A-N-Y-A"

Run each utterance through **two STT systems**. Record results:

#	Utterance	Key entities to extract	System A transcript	System B transcript	A: entities OK?	B: entities OK?
1						
2						
3						
4						
5						

Scoring: Count entity matches (exact or correctly normalized date/phone). **Entity accuracy matters more than word-error rate** for phone agents.

Write **half a page** on which STT you chose and what fails most often (names, dates, IDs, accents).

1.6 TTS hands-on comparison

Write a **2–3 sentence agent greeting** that includes a **recording disclosure**, for example:

> *"Hello, this is the automated scheduling assistant for Acme Clinic. This call may be recorded. How can I help you today?"*

Generate this with **two different voices** (two providers or two voice IDs).

Criterion	Voice A	Voice B
Provider and voice name/ID		
Approx. time to first audio (ms)		
Naturalness (1–5)		
Disclosure easy to understand (1–5)		
Fits your locale/accent (1–5)		

Submit audio files or links. Choose one voice and explain why.

1.7 Tokens, metering, and cost

Phone agents are billed differently per layer:

Layer	Typical billing unit	What drives cost
STT	Audio seconds or minutes	Length of caller speech
LLM	Input + output tokens	Length of prompt + conversation + tool JSON
TTS	Characters, credits, or audio minutes	Length of agent replies

For a **5-minute answered call**, estimate:

Layer	Estimated usage	Approx. cost (show your math)
STT	___ minutes of audio	$
LLM	___ tokens in + ___ out	$
TTS	___ characters or seconds	$
Total per call		**$**

Use public list prices from vendor websites. Label all figures as **estimates**.

1.8 Part 1 checklist — submit all of

- [ ] Half-page problem statement for your domain
- [ ] Model selection matrix (2 options × 3 layers + fallback)
- [ ] STT comparison table (5 utterances × 2 systems)

- [ ] STT analysis paragraph (failure modes)
- [ ] TTS comparison table + 2 audio samples
- [ ] Cost estimate table for a 5-minute call
- [ ] One-page summary: why dictation apps ≠ phone agents

1.9 Part 1 grading criteria

Excellent	Satisfactory	Needs improvement
Data-driven STT/TTS comparison with entity scoring	Lists providers but weak or missing tests	Confuses model roles or skips hands-on work
Clear production vs mock plan	Uses mocks only, no real comparison	No cost or latency discussion

---

Part 2 — Domain Customization (Week 2)

2.1 Learning goals

By the end of Part 2 you should be able to:

- Define terminal outcomes, conversation states, slots, and tools for your use case
- Write a speech-optimized system prompt with guardrails
- Configure an agent with your Part 1 model choices
- Create structured test scenarios with expected outcomes

2.2 Use case specification

Complete the following for **your** domain.

Identity

Field	Your value
Use case name	
Direction	Inbound / Outbound / Both
Locale (language, region)	e.g. en-IN, en-US
Primary user	e.g. patient, customer, lead

Terminal outcomes

Every call must end in exactly one outcome code:

Code	Meaning	Example
`success`	Business task completed	Appointment booked
`escalate`	Transferred or flagged for human	Angry caller, complex case

`user_declined`	User chose not to proceed	“Don’t call again”
`no_answer` / `voicemail`	Outbound only — no human	AMD detected voicemail
`system_fail`	Platform or tool error	CRM down after retries

Your primary success metric (one sentence): e.g. “% of inbound calls that end in `success` with correct slot values.”

Conversation state machine

Draw at least **4 states** and transitions. Example for booking:

```
GREET → COLLECT_INTENT → COLLECT_SLOTS → CONFIRM → BOOK → CLOSING
                            ↓ miss info          ↓ user says no
                         CLARIFY              OFFER_ALTERNATIVES
                            ↓ max turns
                         ESCALATE
```

Fill in your version:

State	Purpose	Max turns in state	Exit to

Slots (information to collect)

Slot name	Required?	Example value	Confirm aloud before acting?

Tools (actions the agent can take)

Tool name	Read-only or side-effect?	Allowed in states	What it does
e.g. `lookup_patient`	Read-only	COLLECT_SLOTS	Find patient by ID or phone
e.g. `book_slot`	Side-effect	CONFIRM, BOOK	Writes appointment to calendar

Side-effect tools change data (book, pay, cancel). They must only run after required slots are collected and, where needed, the user confirms aloud.

2.3 System prompt and guardrails

Write your **system prompt** (maximum 400 words). Rules for phone prompts:

- Use **short sentences** — the caller hears this, not reads it

- No markdown, bullet lists, or JSON in spoken content
- Say **when to escalate** (angry user, out of scope, repeated failure)
- Say **when to call each tool**
- Include **forbidden behaviors** (must not diagnose, must not promise refunds unless tool confirms, etc.)

Disclosure text (spoken at call start):

> *(Your 1–2 sentences here, including recording notice if applicable)*

Must-say phrases (at least 2): e.g. disclosure, confirmation before booking

Must-not-say list (at least 3): things the agent must never claim, e.g. medical diagnosis, guaranteed loan approval, sharing other customers' data

2.4 Model stack binding

Record the models from Part 1 in your agent config:

Layer	Provider	Model / voice ID
STT		
LLM		
TTS		

2.5 Test scenarios (minimum 8)

Each scenario simulates a caller and defines what **must** happen. Create **at least 8** before Part 3 expansion.

Scenario template (copy per scenario):

```yaml
id: your_use_case_happy_001
name: Happy path — books successfully
tags: [happy]
user_script:
  - "Hi, I need to book an appointment"
  - "Dr Sharma, next Tuesday at 4pm"
  - "Yes, that's correct"
expectations:
  expected_outcome: success
  expected_tools: [lookup_doctor, list_slots, book_slot]
  expected_slots:
    doctor: "Dr Sharma"
    datetime: "2026-08-12T16:00"
  must_say:
    - "recorded"          # disclosure keyword
  must_not_say:
    - "guaranteed cure"
```

Required scenario types for Part 2 (at least one each):

Type	What it tests
Happy path	Normal successful completion
Slot fill	User gives info across multiple turns
Tool failure	Backend returns error (slot taken, not found)
Must-not-say	Agent must not utter forbidden phrases
Escalation	User angry or request out of scope → escalate

2.6 Tool failure behavior

Document how your agent should behave when a side-effect tool fails:

Failure	Example	Expected agent behavior
Slot taken	Appointment time no longer available	Apologize, offer next slots, do not pretend success
Not found	Patient ID invalid	Ask to repeat or verify, max 2 retries then escalate
Timeout	CRM slow	Brief hold message, one retry, then escalate

Implement at least **one** failure scenario in your tests with mock tool responses.

2.7 Run and iterate

Run your scenarios in **text simulation mode** (user turns typed as text; speech models bypassed for speed). Iterate on your prompt until **at least 80% of your Part 2 scenarios pass**.

Keep a **prompt changelog**:

Version	Change	Result
v1	Initial prompt	3/8 pass
v2	Added confirm-before-book rule	6/8 pass
v3	Fixed escalation on angry user	7/8 pass

2.8 Part 2 checklist — submit all of

- ☐ Use case specification (identity, outcomes, state machine, slots, tools)
- ☐ System prompt (≤ 400 words) + disclosure + must-say / must-not-say
- ☐ Model stack table
- ☐ ≥ 8 scenario files or equivalent structured definitions
- ☐ Tool failure table + one failure scenario implemented
- ☐ Prompt changelog with pass rates
- ☐ Test run log showing ≥ 80% pass on Part 2 scenarios

2.9 Part 2 grading criteria

Excellent	Satisfactory	Needs improvement
Rich state machine, clear tool policy, original domain	Basic prompt, minimal slots	Copies a generic demo unchanged
Side-effect only after confirm	Tools work but weak guardrails	No tool failure handling

---

Part 3 — Evaluations (Week 3)

3.1 Learning goals

By the end of Part 3 you should be able to:

- Build a **golden set** of test scenarios with explicit expectations
- Define metrics and pass thresholds for your use case
- Run full test suites and interpret failures
- Write a gate policy: what must pass before the agent is “good enough”

3.2 What “eval” means for phone agents

Evaluate the **phone task**, not how eloquent the chatbot sounds.

Primary metric: Task success rate — did the call end with the correct outcome and correct data?

Guardrail metrics (must not regress):

- Zero hits on **must-not-say** phrases
- Zero **unauthorized side effects** (booking without confirm, etc.)
- **100% disclosure compliance** on scenarios that require it

Secondary metrics: Slot accuracy, tool call correctness, average turns on happy paths, latency (if measured).

3.3 Golden scenario set — minimum 15

Expand Part 2 scenarios to **at least 15**. Cover every row you can; mark N/A only with justification:

Category	Min count	What to test
Happy path	3	Straightforward success
Slot fill / missing info	2	User gives data over multiple turns
Tool failure	2	Slot taken, not found, timeout
Escalation / angry user	1	Human handoff
Must-not-say / safety	2	Forbidden claims never spoken
Adversarial injection	1	User says “ignore your instructions…”

Silence / no-input	1	User doesn't respond — reprompt then escalate
Hard entities	2	IDs, dates, phone numbers, spelled names

Naming convention: use clear IDs, e.g. `clinic_happy_001`, `clinic_slot_taken_001`, `clinic_injection_001`.

Scenario fields reference

Every scenario must include:

Field	Description
`id`	Unique name
`user_script`	Ordered list of what the caller says each turn
`expected_outcome`	Terminal code: success, escalate, etc.
`expected_tools`	Tools called, in order (empty if none)
`expected_slots`	Key slot values after the call
`must_say`	Substrings that must appear in agent speech
`must_not_say`	Substrings that must never appear
`tags`	For filtering suites, e.g. happy, tool_fail, injection

3.4 Test suites and thresholds

Define two suites:

Suite	Scenarios included	Pass threshold
Smoke	5 most critical (3 happy + 1 failure + 1 safety)	≥ 90% task success
Full	All 15+ scenarios	≥ 80% task success

Optional **audio suite** (5 scenarios with WAV input or simulated ASR): ≥ 70% if you include audio replay.

Slot normalization rules — document how you compare dates and phones:

Type	Normalization rule	Example
Date	ISO 8601 date	"next Tuesday 4pm" → 2026-08-12T16:00
Phone	E.164 digits only	"98765 43210" → +919876543210
Name	Case-insensitive trim	"dr sharma" → Dr Sharma

3.5 How text simulation works

Text simulation runs your agent logic **without real audio**:

```
For each user utterance in the scenario:
  1. Feed text to the agent as if STT produced it
  2. Agent (LLM + tools) produces reply text
  3. Record tools called, slots filled, transcript

After all turns:
  4. Compare transcript and outcome to expectations
  5. Pass or fail with reasons
```

This is the **default for continuous testing** because it is fast and stable. Audio and live phone tests catch speech recognition issues; text sim catches policy, tool, and logic bugs.

3.6 Run your evals

Execute both suites. Record results in a table:

Metric	Your result	Target	Pass?
Task success rate (full suite)		≥ 80%	
Task success rate (smoke)		≥ 90%	
Critical slot accuracy		≥ 90%	
Tool call correctness		≥ 95%	
Disclosure compliance (tagged scenarios)		100%	
must_not_say violations		0	
Unauthorized side effects		0	
Avg turns (happy paths only)		≤ your limit	

3.7 Failure analysis (required)

For **every failed scenario** in your first full-suite run, document:

1. **Scenario ID** and expected vs actual outcome
2. **Root cause** — prompt gap, wrong tool mock, slot normalization bug, model policy error
3. **Fix applied** — prompt edit, scenario fix, tool behavior change
4. **Re-run result** — pass or fail after fix

Submit evidence that metrics **improved** between first and final run (table or before/after screenshot).

3.8 Gate policy (one page)

Write a policy document answering:

1. **What must pass** before you would allow this agent on a staging phone number?
2. **Which metrics are blocking** vs informational?
3. **Example blocking rules:**
    - Any must_not_say hit → block

    - Disclosure missing on tagged scenarios → block
    - Side-effect tool without confirmation → block
    - Full suite task success below 80% → block
4. **What you would test next** before production (audio replay, live calls, load test).

3.9 Optional — audio replay

If you have time, add **5 audio scenarios** (WAV files or simulated transcripts from noisy phone audio). Test whether STT errors cause task failures. Write a short **ASR risk memo**: which entities fail most and mitigations (repeat aloud, spell mode, DTMF fallback).

3.10 Part 3 checklist — submit all of

- [ ] Scenario catalog listing all 15+ scenarios with tags
- [ ] Smoke + full suite definitions and thresholds
- [ ] Normalization rules for dates, phones, names
- [ ] Results table (Section 3.6) from final run
- [ ] Failure analysis for all first-run failures + fixes
- [ ] One-page gate policy
- [ ] Before/after metrics showing improvement

3.11 Part 3 grading criteria

Excellent	Satisfactory	Needs improvement
15+ scenarios, thresholds met, thorough failure mining	15 scenarios but weak analysis or below threshold without plan	Fewer than 15 scenarios or no metrics
Zero must_not_say violations on final run	One violation explained and fixed	Safety scenarios ignored

---

Final Portfolio

Combine into one PDF

1. **Cover page** — your name, use case title, date
2. **Part 1 report** — models (Sections 1.3–1.7)
3. **Part 2 report** — domain customization (Sections 2.2–2.7)
4. **Part 3 report** — evaluations (Sections 3.3–3.8)
5. **Appendix** — scenario files, test logs, audio samples

Final rubric (100 points)

Area	Points	Criteria
Models	30	Comparison tables, entity-focused STT test, TTS samples, cost estimate
Domain	35	Complete spec, prompt quality, ≥8 then 15+ scenarios, tool policy

Evals	35	Metrics, failure analysis, gate policy, demonstrated improvement
Total	**100**	

Academic integrity

- Cite vendor documentation for pricing and API limits.
- Disclose if generative AI assisted your **written** report.
- Your use case and prompts must be **original** — do not submit an unchanged generic booking demo.
- Never commit API keys or secrets to a public repository.

Quick reference — scenario categories

Use this when building your golden set:

Conversation / task

- Happy path → success
- Multi-turn slot fill
- User declines → soft end
- Max turns reached → escalate
- Wrong person / wrong number

Speech / audio (optional)

- Background noise
- Accent or fast speech
- Spelled ID or name
- Dates and amounts
- User interrupts agent mid-sentence (barge-in)

Tools / systems

- Tool timeout
- Slot already taken
- Record not found
- Side effect blocked until verified

Safety / compliance

- Disclosure required on answer
- Forbidden claims (must_not_say)
- Prompt injection via speech (“ignore instructions”)

Quick reference — model selection scorecard

When choosing STT / LLM / TTS for production, weight these factors:

Factor	Weight	Notes
Task success on your golden set	High	Most important

Critical entity accuracy	High	Names, IDs, dates, amounts
Turn latency (p50 / p95)	High	Target ≤ 1.2s median for good UX
Cost per answered minute	Medium	STT + LLM tokens + TTS
Language and accent fit	High	Test on your locale’s phone audio
Operational complexity	Medium	API vs self-host, fallbacks, monitoring

Publish a one-page decision record and **store model IDs on every test run** so you can compare results over time.

End of assignment.